

1. Spring-Actuator

1.1. Présentation de "spring-actuator"

L'extension standard "spring-actuator" permet de récupérer automatiquement des indications (métriques) sur le fonctionnement interne d'une application spring-boot .

Spring-actuator permet très concrètement d'intégrer dans l'application développée *tout un tas de micros-services REST techniques/annexes* permettant de *récupérer certaines métriques (variables d'environnement , consommation mémoire , ...)* .

Exemple :

http://localhost:8181_ou_autre/my-spring-boot-app/**actuator/health**

retourne {"status":"UP"} quand l'application fonctionne bien
ou bien {"status":"DOWN"} quand l'application ne fonctionne pas bien

En production/exploitation, Ceci peut être très utile pour effectuer un suivi/pilotage de l'application et surveiller son bon fonctionnement .

En développement/debug , spring-actuator peut grandement aider à trouver certains réglages permettant d'optimiser les performances de l'application .

1.2. Mise en oeuvre de spring-actuator

dans pom.xml :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

dans **application.properties**

```
#actuators:
#management.endpoints.web.exposure.include=*
management.endpoints.web.exposure.include=health,info,env,metrics
```

Important : * en dev seulement ,

En production , trop de mesures = application ralentie !!!

Et si besoin dans WebSecurityConfig ou ... :

```
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {
    ....
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/actuator/**").permitAll()
                .anyRequest().authenticated();
    }
    ...}
}
```

1.3. URLs des "endpoints" de spring actuator :

http://localhost:8181_ou_autre/my-spring-boot-app/actuator

renvoie au format JSON la liste des urls précises des "endpoints" de spring-actuator :

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8181/spring-boot-backend/actuator",
      "templated": false
    },
    "beans": {
      "href": "http://localhost:8181/spring-boot-backend/actuator/beans",
      "templated": false
    },
    "caches-cache": {
      "href": "http://localhost:8181/spring-boot-backend/actuator/caches/{cache}",
      "templated": true
    },
    "caches": {
      "href": "http://localhost:8181/spring-boot-backend/actuator/caches",
      "templated": false
    },
    "health": {
      "href": "http://localhost:8181/spring-boot-backend/actuator/health",
      "templated": false
    },
    "health-path": {
      "href": "http://localhost:8181/spring-boot-backend/actuator/health/{*path}",
      "templated": true
    },
    "info": {
      "href": "http://localhost:8181/spring-boot-backend/actuator/info",
      "templated": false
    },
    "conditions": {
      "href": "http://localhost:8181/spring-boot-backend/actuator/conditions",
      "templated": false
    },
    "configprops": {
```

```

    "href": "http://localhost:8181/spring-boot-backend/actuator/configprops",
    "templated": false
  },
  "env": {
    "href": "http://localhost:8181/spring-boot-backend/actuator/env",
    "templated": false
  },
  "env-toMatch": {
    "href": "http://localhost:8181/spring-boot-backend/actuator/env/{toMatch}",
    "templated": true
  },
  "loggers": {
    "href": "http://localhost:8181/spring-boot-backend/actuator/loggers",
    "templated": false
  },
  "loggers-name": {
    "href": "http://localhost:8181/spring-boot-backend/actuator/loggers/{name}",
    "templated": true
  },
  "heapdump": {
    "href": "http://localhost:8181/spring-boot-backend/actuator/heapdump",
    "templated": false
  },
  "threaddump": {
    "href": "http://localhost:8181/spring-boot-backend/actuator/threaddump",
    "templated": false
  },
  "metrics-requiredMetricName": {
    "href": "http://localhost:8181/spring-boot-backend/actuator/metrics/{requiredMetricName}",
    "templated": true
  },
  "metrics": {
    "href": "http://localhost:8181/spring-boot-backend/actuator/metrics",
    "templated": false
  },
  "scheduledtasks": {
    "href": "http://localhost:8181/spring-boot-backend/actuator/scheduledtasks",
    "templated": false
  },
  "mappings": {
    "href": "http://localhost:8181/spring-boot-backend/actuator/mappings",
    "templated": false
  }
}

```

<http://localhost:8181/my-spring-boot-app/actuator/metrics/http.server.requests>

retourne

```

{
  "name": "http.server.requests",
  "description": null,
  "baseUnit": "seconds",
  "measurements": [
    {
      "statistic": "COUNT",
      "value": 95
    },
    {
      "statistic": "TOTAL_TIME",
      "value": 4.648203600000001
    }
  ]
}

```

```

    },
    {
      "statistic": "MAX",
      "value": 0.7233778
    }
  ],
  ....
}

```

avec dans **application.properties**

```

#management.endpoint.env.show-values=WHEN_AUTHORIZED
management.endpoint.env.show-values=ALWAYS

```

<http://localhost:8181/my-spring-boot-app/actuator/env>

retourne

```

{
  "activeProfiles": [
    "embeddedDb",
    "reInit",
    "appDbSecurity"
  ],
  "propertySources": [
    {
      "name": "server.ports",
      "properties": {
        "local.server.port": {
          "value": 8181
        }
      }
    },
    {
      "name": "servletContextInitParams",
      "properties": {}
    },
    {
      "name": "systemProperties",
      "properties": {
        "sun.desktop": {
          "value": "windows"
        },
        "awt.toolkit": {
          "value": "sun.awt.windows.WToolkit"
        },
        "java.specification.version": {
          "value": "11"
        },
        "sun.cpu.isalist": {
          "value": "amd64"
        },
        "sun.jnu.encoding": {
          "value": "Cp1252"
        },
        "java.class.path": {
          "value": "D:\\tp\\local-git-mycontrib-repositories\\env-ic-my-java-rest-app\\target\\classes;...."
        }
      }
    }
  ]
}

```

```

}, ...,
    "java.vendor.url": {
        "value": "http://java.oracle.com/"
    },
    "catalina.useNaming": {
        "value": "false"
    },
    "user.timezone": {
        "value": "Europe/Paris"
    },
    "os.name": {
        "value": "Windows 10"
    },
    "java.vm.specification.version": {
        "value": "11"
    },
    "sun.java.launcher": {
        "value": "SUN_STANDARD"
    },
    "user.country": {
        "value": "FR"
    }, ...
"JAVA_HOME": {    "value": "C:\\Program Files\\Java\\jdk-11.0.4",
                  "origin": "System Environment Property \\\"JAVA_HOME\\\""
    }, ...

```

et `http://localhost:8181/my-spring-boot-app/actuator/env/user.timezone`
retourne

```

{
  "property": {
    "source": "systemProperties",
    "value": "Europe/Paris"
  }, ...
}

```

...

1.4. Paramétrages (application.properties) de `actuator/info`

En ajoutant dans `app.properties`

```

#app infos for actuator/info  if management.info.env.enabled=true
management.info.env.enabled=true
#info.app.MY_APP_PROP_NAME=ValeurQuiVaBien
info.app.name=spring-boot-backend
info.app.description=appli spring-boot , backend with rest-api (micro services)

```

alors l'actuator prédéfini **actuator/info** renvoi

```
{
  "app": {
    "name": "spring-boot-backend",
    "description": "appli spring-boot , backend with rest-api (micro services)"
  }
}
```

1.5. Codage d'un "health indicator" spécifique

Avec dans application.properties :

```
management.endpoint.health.show-details=ALWAYS
```

et avec

```
@Component
public class MyHealthIndicator implements HealthIndicator {

    long checkSomething() {
        return 1;
    }

    @Override
    public Health health() {
        long result = checkSomething();
        if (result <= 0) {
            return Health.down().withDetail("Something Result", result).build();
        }
        return Health.up().withDetail("isXxxOk", true)
            .withDetail("isYyyOk", true).build();
    }
}
```

```
▼ my:
  status: "UP"
  ▼ details:
    isXxxOk: true
    isYyyOk: true
```

est retourné par **actuator/health** .

Autre variante possible : ... implements **ReactiveHealthIndicator**

2. avec io.micrometer

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-core</artifactId>
</dependency>
```

Au plus simple :

```
import io.micrometer.observation.annotation.Observed;
...
@Service
public class ServiceXyz{
...
    @Observed(name = "searchById")
    //.../actuator/metrics/searchById
    public Compte searchById(Long aLong) throws EntityNotFoundException {
        ...
    }
}
```

type d'observation via une url finissant par /actuator/metrics/searchById

```
{ "name": "searchById", "baseUnit": "seconds",
  "measurements": [
    { "statistic": "COUNT", "value": 6.0 },
    { "statistic": "TOTAL_TIME", "value": 0.0253145 },
    { "statistic": "MAX", "value": 0.0140763 } ],
  "availableTags": [
    { "tag": "method", "values": [ "searchById" ] },
    { "tag": "error", "values": [ "none" ] },
    { "tag": "class", "values": [ "tp.appliSpring.bank.core.service.direct.ServiceXyz" ] }
  ]
}
```

6 appels avec un temps total de 0.0253145s soit une moyenne de $0.0253145 / 6 = 0.004219$ s

Micrometer intégré dans spring :

Counter : increment only

Gauge : increment or decrement

Timer : execution times (max ,total ,avg , ...)

`Observation.observe( ) -> { code_of_duration_to_observe }`

build automatically a sort of Timer (like `@Observe()` on top of a whole method)

`//.../actuator/metrics/_name_of_metric_or_observation_`

Mesure personnalisée de type **compteur** et observation de **temps d'exécution**:

```
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.observation.Observation;
import io.micrometer.observation.ObservationRegistry;

@Service
public class ServiceXyz ... {

    private CompteRepository daoCompte;
    private ObservationRegistry observationRegistry;
    private final Counter accountCreatedCounter;

    @Autowired
    public ServiceXyz(CompteRepository daoCompte,
                     ObservationRegistry observationRegistry,
                     MeterRegistry registry){

        ...
        this.observationRegistry=observationRegistry;
        this.accountCreatedCounter=Counter.builder("accounts.created")
            .description("Number of accounts created")
            .register(registry);
        //.../actuator/metrics/accounts.created
    }

    public Compte create(Compte obj) {
        //.../actuator/metrics/CREATED_ACCOUNT_STAT
        return Observation
            .createNotStarted("CREATED_ACCOUNT_STAT", observationRegistry)
            //lowCardinalityKeyValue("xxx", "yyyy") //bounded (non infinite) set size
            .observe() -> {
                this.accountCreatedCounter.increment();
                // Execute business logic here
                Compte compte = .... ;
                /*
                try {
                    .sleep(1500);
                } catch (InterruptedException e) {
                    throw new RuntimeException(e);
                }
                */
                //System.out.println("a new account was added : " + obj);
                return compte;
            });
    }
}
```


Type de résultats :

//.../actuator/metrics/*accounts.created*

```
{ "name": "accounts.created", "description": "Number of accounts created",
  "measurements": [ { "statistic": "COUNT", "value": 4.0 }, "availableTags": [] ] }
```

et //.../actuator/metrics/CREATED_ACCOUNT_STAT

produit un résultat similaire à celui produit par @Observe()

Configuration quelquefois utile :

@Configuration

public class ObservabilityHandlerConfig {

```
    private static final Logger log = LoggerFactory.getLogger(ObservabilityHandlerConfig.class);
```

```
    /* // ObservationRegistry
```

```
    @Bean
```

```
    ObservationRegistry observationRegistry() {
```

```
        ObservationRegistry myObservationRegistry = ObservationRegistry.create();
```

```
        //myObservationRegistry.observationHandler(new
```

```
        ObservationTextPublisher(System.out::println)); //for basic local display
```

```
        return myObservationRegistry;
```

```
    } */
```

```
    @Bean
```

```
    public ObservationHandler<Observation.Context> observationTextPublisher() {
```

```
        return new ObservationTextPublisher(log::info); //to display observation in logger
```

```
    }
```

```
    /* // To have the @Observed support we need to register this aspect (it's seems already done
    in spring boot 3 but not spring boot 4) */
```

```
    @Bean
```

```
    ObservedAspect observedAspect(ObservationRegistry observationRegistry) {
```

```
        return new ObservedAspect(observationRegistry);
```

```
    }
```

```
}
```

Extensions classiques :

Prometheus - an open-source systems monitoring and alerting toolkit

Grafana - an open source data visualization and monitoring solution

article sur le sujet :

<https://medium.com/@aleksanderkolata/spring-boot-micrometer-prometheus-and-grafana-how-to-add-custom-metrics-to-your-application-712c6f895f6b>

3. Récupération et affichage distant des mesures

3.1. Ancien SpringBootAdmin (extension de de.codecentric)

- **SpringBootAdmin** était une extension spring de " de.codecentric" qui permettait de mettre assez facilement en oeuvre un serveur de surveillance/administration de certaines applications springBoot.
- *Cette application n'est aujourd'hui plus maintenue et est considérée comme obsolète*

3.2. Prometheus + Grafana

Les évolutions récentes de spring sur la partie "Observability" ont clairement pris la direction d'un dialogue avec les standards du devOps , à savoir pouvoir communiquer avec Prometheus ou des outils équivalents .

En ajoutant dans **pom.xml** une dépendance de ce genre :

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
  <version>1.16.7</version>
</dependency>
```

et en configurant **application.properties** de cette manière :

```
management.endpoint.prometheus.enabled=true
management.endpoints.web.exposure.include=health,info,env,metrics,prometheus
```

On peut alors déclencher une url de ce type

<http://localhost:8080/appliSpring/actuator/prometheus>

permettant de **lister** tout un tas de "*query*" *possibles* au niveau de l'outil "*prometheus*" (avec des exemples de valeurs à l'instant t).

```
jdbc_connections_min{name="dataSource"} 10.0
...
jvm_memory_max_bytes{area="heap",id="G1 Old Gen"} 4.060086272E9
process_uptime_seconds 26.09
```

Pour installer et lancer Prometheus (avec ou sans le complément Grafana) , on peut éventuellement s'appuyer sur un fichier **docker-compose.yml** de de type :

```
services:
  prometheus:
    image: prom/prometheus:v3.13.3
    container_name: prometheus
    ports:
      - "9090:9090"
    extra_hosts:
      #- "host.docker.internal:host-gateway"
      - "host.docker.internal:172.18.192.1"
      #avec 172.18.192.1 resultat de ip route show | grep -i default | awk '{ print $3}' sous WSL
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana:13.0.8
    container_name: grafana
    ports:
      - "3000:3000"
    restart: unless-stopped
    volumes:
      - ./grafana/provisioning/datasources:/etc/grafana/provisioning/datasources
```

NB : host.docker.internal vaut exceptionnellement ici 172.18.192.1 au sein d'un environnement WSL devant accéder à l'host "windows 11" vu avec l'adresse IP 172.18.192.1

Le fichier fondamental prometheus/**prometheus.yml**

permet de préciser l'adresse de l'application SpringBoot à surveiller/contrôler :

```
scrape_configs:
  - job_name: 'MyAppMetrics'
    #metrics_path: '/actuator/prometheus'
    metrics_path: '/appliSpring/actuator/prometheus'
    scrape_interval: 3s
    static_configs:
      - targets: ['host.docker.internal:8181']
        labels:
          application: 'My Spring Boot Application'
```

Pour un meilleur affichage avec l'outil complémentaire grafana , on peut paramétrer la communication entre grafana et prometheus via un fichier de de genre

grafana/provisioning/datasources/automatic.yml

```
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
isDefault: true
```

Pour démarrer le tout dans un environnement compatible docker (ex: linux ou WSL sous windows) on peut lancer la commande

```
docker compose up -d
```

et vérifier via

```
docker container ls
```

et pour ultérieurement tout arrêter

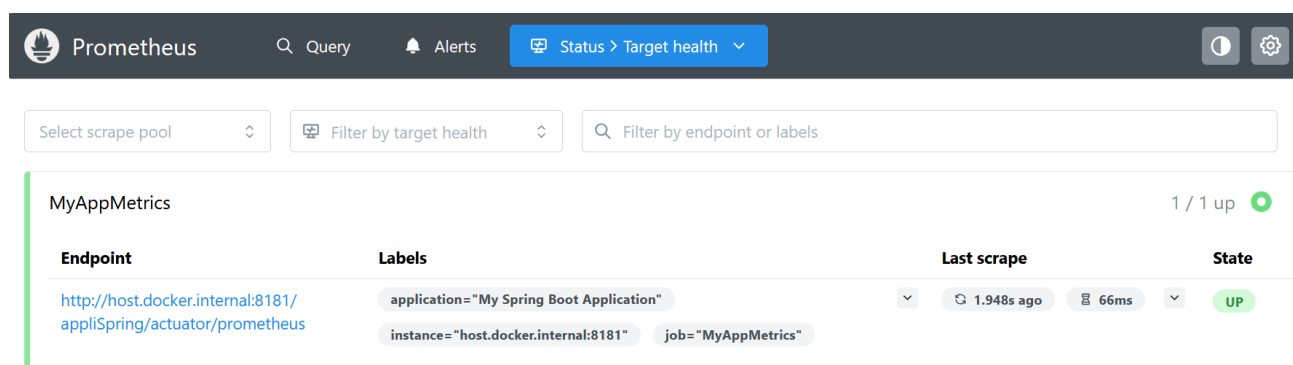
```
docker compose down
```

Via un navigateur web , avec l'url **localhost:3000** on accède à la console "**grafana**" (avec admin/admin comme login par défaut) . Ceci permet d'effectuer de beaux affichages lorsque l'on connaît bien le produit .

L'essentiel réside plutôt dans l'outil "**Prometheus**" qui récolte régulièrement les mesures/métriques provenant de l'application "springBoot".

Via un navigateur web , avec l'url **localhost:9090** on accède à la console "**prometheus**"

Il est conseillé de vérifier la communication avec l'appli spring Boot de cette façon :



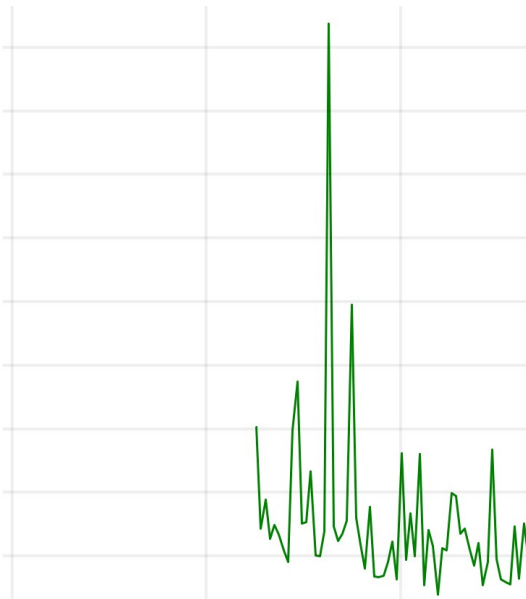
The screenshot shows the Prometheus web interface. At the top, there's a navigation bar with 'Prometheus', 'Query', 'Alerts', and 'Status > Target health'. Below this, there are filters for 'Select scrape pool', 'Filter by target health', and 'Filter by endpoint or labels'. The main content area displays a table of targets. The first target is 'MyAppMetrics', which is '1 / 1 up' (indicated by a green dot). The table has columns for 'Endpoint', 'Labels', 'Last scrape', and 'State'. The 'Endpoint' column shows 'http://host.docker.internal:8181/appliSpring/actuator/prometheus'. The 'Labels' column shows 'application="My Spring Boot Application"', 'instance="host.docker.internal:8181"', and 'job="MyAppMetrics"'. The 'Last scrape' column shows '1.948s ago' and '66ms'. The 'State' column shows 'UP' with a green dot.

Dans un partie "query" on peut saisir des requêtes prometheus possibles (une de celles qui sont listées via appliSpring/actuator/prometheus)



The screenshot shows the Prometheus web interface with the 'Query' tab selected. The top navigation bar includes 'Prometheus', a moon icon, a gear icon, a book icon, and a hamburger menu icon. Below the navigation bar, there is a large input field for queries. The input field contains the text '>_ system_cpu_usage'. To the right of the input field is a blue button labeled 'Execute'.

Puis via "execute" et l'onglet "table" ou "graph" on peut visualiser les métriques obtenues :



Attention si observer personnalisé via `@Observer(name="xxx.yyy.zzz")` dans spring alors le nom vu par prometheus est `xxx_yyy_zzz`

Exemple pour `@Observer(name="serviceCompte_searchAll")`

`http://localhost:8181/appliSpring/actuator/metrics/serviceCompte_searchAll`

`http://localhost:8181/appliSpring/actuator/prometheus` partial return list:

```
serviceCompte_searchAll_seconds_count{class="tp.appliSpring.bank.core.service.impl.ServiceCompteImpl",error="none",method="searchAll"}
serviceCompte_searchAll_seconds_sum{class="tp.appliSpring.bank.core.service.impl.ServiceCompteImpl",error="none",method="searchAll"}
serviceCompte_searchAll_seconds_max{class="tp.appliSpring.bank.core.service.impl.ServiceCompteImpl",error="none",method="searchAll"}
```

Et ces requêtes spécifiques/personnalisées sont exploitables/affichables au sein de l'outil "prometheus" .

3.3. opentelemetry

Evolution très récente de spring7 / springBoot 4 :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-opentelemetry</artifactId>
</dependency>
```

Encore plus souple , paramétrable mais très récent (à approfondir dès que possible).